

Memetic Algorithm Performance Evaluation for the Quadratic Assignment Problem

Olivia-Alexandra Gavril

15 December 2025

Abstract

Augmenting genetic algorithms with local search heuristics is an effective strategy for solving difficult combinatorial optimization problems. In this paper, a memetic algorithm for the *Quadratic Assignment Problem* (QAP) is presented, combining a *Genetic Algorithm* for global exploration with *Simulated Annealing* as a local refinement procedure. The local search is applied to high-quality individuals in order to intensify the search and improve convergence toward better solutions. The proposed approach is evaluated on a set of benchmark instances from QAPLIB, including problems with dimensions ranging from 12 up to 256. Experimental results show that the algorithm achieves optimal solutions for several instances with known optima and produces competitive solutions for large-scale instances where the optimal solution is not known. Overall, the results suggest that the proposed memetic algorithm offers a reliable and scalable solution for the Quadratic Assignment Problem.

1 Introduction

The *Quadratic Assignment Problem* (QAP) is an optimization problem that deals with assigning a set of facilities to a set of locations, considering the pairwise distances and flows between them[6]. The objective is to find an assignment that minimizes the total cost resulting from the interaction between facilities and the distances separating their assigned locations[8]. QAP is classified as NP-hard and is difficult to solve for large dimensions [8].

In this work, we address QAP using a *Memetic Algorithm*, which combines a Genetic Algorithm (GA) with a local search procedure based on Simulated Annealing (SA). The genetic algorithm is responsible for global exploration of the solution space through selection, crossover, and mutation, while simulated annealing is applied as a local refinement method to improve promising individuals. The local search is applied iteratively to the best individuals of each generation in order to enhance solution quality and improve convergence.

Since the evaluation of solutions and local search procedures are computationally expensive, particular attention is given to reducing execution time. Parallel evaluation of the population is employed, and SA is applied selectively only to a small number of high-quality individuals. These design choices allow the algorithm to scale to larger instances while maintaining acceptable execution times.

The proposed algorithm is evaluated on benchmark instances from QAPLIB, covering small, medium, and large problem sizes, ranging from 12 to 256 facilities and locations. The results show that the algorithm attains optimal solutions on instances with known optima and achieves competitive results on larger, more challenging instances.

2 Methods and Implementation

2.1 QAP: Mathematical Formulation and Solution Representation

In this work, the Quadratic Assignment Problem is addressed using a permutation-based representation. Let $I = \{1, \dots, n\}$ denote the set of objects and locations. A solution is represented by a permutation

$$\psi : I \rightarrow I,$$

where $\psi(i)$ denotes the location assigned to object i [9]. This representation satisfies the assignment constraints, since each object is assigned to exactly one location and each location appears exactly once in the permutation.

The problem is defined by two $n \times n$ matrices: the flow matrix $B = (b_{ij})$, where b_{ij} represents the interaction between objects i and j , and the distance matrix $A = (a_{kl})$, where a_{kl} denotes the distance between locations k and l . The objective is to find a permutation ψ that minimizes the total assignment cost

$$f(\psi) = \sum_{i=1}^n \sum_{j=1}^n b_{ij} a_{\psi(i)\psi(j)}.$$

[8]

The term $b_{ij} a_{\psi(i)\psi(j)}$ corresponds to the cost contribution of assigning object i to location $\psi(i)$ and object j to location $\psi(j)$ [9].

2.2 Genetic Algorithm

The Genetic Algorithm (GA) represents the global search component of the proposed memetic framework.

Solution representation Each individual is represented as a permutation of length n , where the position of each element corresponds to the location assigned to a facility. This representation naturally enforces the assignment constraints of the QAP and ensures that all generated solutions are feasible.

Initial population The initial population consists of randomly generated permutations, which ensure a diverse set of feasible solutions and provide a solid starting point for the evolutionary search process.

Selection The selection operation is a fundamental process in Genetic Algorithms (GAs) that determines which individuals from the current population will contribute to the creation of the next generation.[10] In our work, we chose a Tournament Selection mechanism. In this method, a subset of individuals is randomly chosen from the population, and the one with the highest fitness is selected for reproduction. This process is repeated until the required number of individuals is selected [10]. This strategy introduces selective pressure while preserving population diversity.

Crossover Crossover is performed using the Ordered Crossover (OX) operator, which is well suited for permutation based representations. Two cut positions are selected at random, and the segment between them is copied from the first parent into the offspring. The remaining positions are then filled according to the order of elements in the second parent, skipping those already present, thereby preserving relative ordering and ensuring that the resulting offspring is a valid permutation.

Mutation Mutation is implemented as a swap operation between two randomly selected positions in the permutation. To balance exploration and exploitation, the mutation rate is adjusted dynamically based on the search progress. When the algorithm stagnates, the mutation probability is increased to promote exploration, while it is reduced when improvements are observed.

Elitism An elitism mechanism is employed by copying the best individual of each generation directly into the next generation.

The genetic algorithm is executed for a fixed number of generations, after which the best solution found is returned. Selected high-quality individuals are further refined using Simulated Annealing as part of the memetic framework.

2.3 Simulated Annealing algorithm

General principle Simulated Annealing is a probabilistic optimization algorithm inspired by the physical process of annealing in metallurgy, where a material is heated to a high temperature and then slowly cooled in order to reach a stable structure with minimal energy. In optimization, this analogy translates to exploring a solution space in order to find the optimal solution of an instance [2].

Control parameters The behavior of the Simulated Annealing algorithm is mainly determined by a set of control parameters that guide its search dynamics. The most important among these are the *initial temperature* (T_0), the *cooling rate* (α), and the *number of iterations* performed at each temperature level.

Initial temperature estimation The initial temperature T_0 is estimated using a short calibration phase, during which several random transitions are generated from the initial solution. For each transition, the change in the objective function Δf is computed. The average of all positive changes $\overline{\Delta f^+}$ is then used to estimate the initial temperature according to the formula:

$$T_0 = \frac{\overline{\Delta f^+}}{\ln \left(\frac{m_2}{m_2 p_0 - m_1 (1 - p_0)} \right)}$$

[2]

where:

- m_1 – number of improving transitions ($\Delta f \leq 0$)
- m_2 – number of worsening transitions ($\Delta f > 0$)
- p_0 – initial acceptance rate
- $\overline{\Delta f^+}$ – mean increase in the objective function for worsening transitions

Acceptance criterion and cooling schedule At the start of the process, worse solutions are accepted with a higher probability, allowing the algorithm to explore the search space more freely before gradually focusing on better regions. The probability of accepting a non-improving solution is defined by the *Metropolis acceptance criterion*:

$$P = e^{-\frac{\Delta f}{T}},$$

where Δf represents the change in the objective function.

The temperature is reduced following a geometric cooling scheme, expressed as $T_{k+1} = \alpha T_k$, where $0.8 < \alpha < 0.99$ [2]. Higher values of α lead to a slower cooling process, promoting exploration at the cost of increased computation time.

Neighborhood generation and efficiency At each temperature level, a fixed number of neighboring solutions are generated and evaluated. In this implementation, neighboring solutions are obtained by *swapping* two positions in the permutation representing the current assignment. To reduce computational time, the objective function is not fully recomputed after each swap. Instead, only the *cost difference* induced by the exchanged positions is calculated.

Stopping criteria The algorithm stops when the temperature falls below a predefined threshold or when no improvement is observed for a fixed number of consecutive temperature levels.

3 Experiments and Results

3.1 Experimental Setup

Experiments were performed on the following QAPLIB [7] benchmark instances: **had14**, **esc16a**, **chr12a**, **nug30**, **lipa30b**, **tho30**, **esc64a**, **sko81**, **tai100a**, and **tai256c**. All instances were evaluated over 30 independent runs.

A fixed *population size* of 200 individuals was used throughout all experiments. The algorithm was run for a maximum of 2000 *generations*. Crossover was applied with a probability of 0.4. The mutation rate was adaptively adjusted to increase diversity during stagnation: 0.15 by default, 0.20 after 60 stagnating generations, and 0.30 after 120 stagnating generations.

Computational efficiency was improved by updating the solution cost incrementally in Simulated Annealing rather than recomputing it entirely. Furthermore, OpenMP parallelization was applied to fitness evaluation and offspring generation, allowing multiple individuals to be processed simultaneously.

3.2 Experimental Results

The tested instances are grouped according to their size. Small instances include chr12a, had14, and esc16a. The medium size instances consist of nug30, lipa30b, tho30, and esc64a, while sko81, tai100a, and tai256c represent larger scale problems.

Small instances

For all small instances, the proposed algorithm consistently reached the known optimal solution in all runs, as reflected by a zero standard deviation. This indicates that the method is both correct and robust for problems with small dimensions.

Instance name	chr12a
Known optimum	9552
Min. Value	9552
Max. Value	9552
Mean	9552
Standard Deviation	0

Table 1: Performance statistics of the proposed memetic algorithm on chr12a

chr12a	Min. Value	Max. Value	Mean	Standard Deviation
Running Time	14.8383	15.7063	15.1278	0.1990

Table 2: Running time statistics for chr12a (in seconds)

Instance name	had14
Known optimum	2724
Min. Value	2724
Max. Value	2724
Mean	2724
Standard Deviation	0

Table 3: Performance statistics of the proposed memetic algorithm on had14

had14	Min. Value	Max. Value	Mean	Standard Deviation
Running Time	16.3095	17.8239	16.8002	0.3210

Table 4: Running time statistics for had14 (in seconds)

Instance name	esc16a
Known optimum	68
Min. Value	68
Max. Value	68
Mean	68
Standard Deviation	0

Table 5: Performance statistics of the proposed memetic algorithm on esc16a

esc16a	Min. Value	Max. Value	Mean	Standard Deviation
Running Time	17.6183	19.0424	17.9417	0.3291

Table 6: Running time statistics for esc16a (in seconds)

Medium instances

For medium-sized instances, the algorithm achieved optimal or near-optimal solutions. In particular, the optimal solution was consistently found for lipa30b and esc64a, while nug30 and tho30 exhibit small variability across runs. This behavior can be explained by the fact that nug30 and tho30 are harder instances, where many different assignments lead to similarly good solutions, causing slight differences between runs.

Instance name	lipa30b
Known optimum	151426
Min. Value	151426
Max. Value	151426
Mean	151426
Standard Deviation	0

Table 7: Performance statistics of the proposed memetic algorithm on lipa30b

lipa30b	Min. Value	Max. Value	Mean	Standard Deviation
Running Time	33.2312	38.9165	35.1989	1.3767

Table 8: Running time statistics for lipa30b (in seconds)

Instance name	tho30
Known optimum	149936
Min. Value	149936
Max. Value	150862
Mean	150159.8
Standard Deviation	298.82

Table 9: Performance statistics of the proposed memetic algorithm on tho30

tho30	Min. Value	Max. Value	Mean	Standard Deviation
Running Time	34.1281	38.7282	35.7466	1.2585

Table 10: Running time statistics for tho30 (in seconds)

Instance name	nug30
Known optimum	6124
Min. Value	6124
Max. Value	6148
Mean	6130.7333
Standard Deviation	7.2108

Table 11: Performance statistics of the proposed memetic algorithm on nug30

nug30	Min. Value	Max. Value	Mean	Standard Deviation
Running Time	36.8761	76.5147	47.3905	7.2532

Table 12: Running time statistics for nug30 (in seconds)

Instance name	esc64a
Known optimum	116
Min. Value	116
Max. Value	116
Mean	116
Standard Deviation	0

Table 13: Performance statistics of the proposed memetic algorithm on esc64a

esc64a	Min. Value	Max. Value	Mean	Standard Deviation
Running Time	138.206	152.573	145.9252	3.6443

Table 14: Running time statistics for esc64a (in seconds)

Large instances

In cases involving large instances, where the search space is computationally intractable, the algorithm produced viable feasible solutions. While a pronounced increase in solution variability was apparent with problem growth, the overall consistency of the results indicates that the approach retains its utility when applied to complex problem scales.

Instance name	sko81
feas.sol.	90998
Min. Value	91072
Max. Value	91236
Mean	91146.9333
Standard Deviation	40.4517

Table 15: Performance statistics of the proposed memetic algorithm on sko81

sko81	Min. Value	Max. Value	Mean	Standard Deviation
Running Time	208.95257	213.53899	211.5925	1.1433

Table 16: Running time statistics for sko81 (in seconds)

Instance name	tai100a
feas.sol.	21052466
Min. Value	21470766
Max. Value	21593662
Mean	21533262.0667
Standard Deviation	28513.3439

Table 17: Performance statistics of the proposed memetic algorithm on tai100a

tai100a	Min. Value	Max. Value	Mean	Standard Deviation
Running Time	233.79302	307.21384	302.4242	13.0390

Table 18: Running time statistics for tai100a (in seconds)

Instance name	tai256c
feas.sol.	44759294
Min. Value	44822748
Max. Value	44895758
Mean	44856402.8
Standard Deviation	16041.9293

Table 19: Performance statistics of the proposed memetic algorithm on tai256c

tai256c	Min. Value	Max. Value	Mean	Standard Deviation
Running Time	875.33406	932.27774	894.4424	11.9488

Table 20: Running time statistics for tai256c (in seconds)

Overall, the experimental outcomes suggest that the proposed memetic algorithm offers a viable and generally reliable method for the Quadratic Assignment Problem. For small instances, the method found the known optimal solutions, demonstrating its convergence ability. For the larger, more complex instances where the global optimum is computationally out of reach, the algorithm successfully produced reasonable and competitive feasible solutions.

The observed behavior of the algorithm can be attributed to a clear balance between exploration and exploitation: the Genetic Algorithm explores the search space, while Simulated Annealing intensifies the search around promising solutions.

4 Conclusions

This paper introduces and experimentally evaluates a memetic algorithm designed for the Quadratic Assignment Problem. The approach effectively combines the global search capabilities of a Genetic

Algorithm with the local refinement power of Simulated Annealed, thereby aiming to achieve an optimal balance between the exploration of the search space and the exploitation of high-quality solutions.

The algorithm was tested on a diverse set of benchmark instances from QAPLIB, ranging from small to large problem sizes. For small instances, the proposed method consistently reached the known optimal solutions in all runs, confirming the correctness of the implementation. For instances of medium complexity, optimal or near optimal solutions were obtained, with limited variability across runs. For larger instances, where optimal solutions are difficult or impractical to reach, the algorithm produced competitive feasible solutions within reasonable computational time.

Overall, the results suggest that the proposed memetic algorithm provides a reliable strategy for solving Quadratic Assignment Problem instances of varying sizes. The complementary interaction between global exploration and local refinement enables the algorithm to maintain consistent behavior as problem complexity increases.

Resources and references

- [1] Lect. dr. Eugen Croitoru. *Genetic Algorithms*.
Available at: <https://profs.info.uaic.ro/eugen.croitoru/teaching/ga/>
- [2] Daniel Delahaye, Supatcha Chaimatanan, Marcel Mongeau. *Simulated annealing: From basics to applications*.
Gendreau, Michel; Potvin, Jean-Yves. *Handbook of Metaheuristics*, 272, Springer, pp. 1–35.
ISBN 978-3-319-91085-7, 2019, International Series in Operations Research & Management Science (ISOR), 978-3-319-91086-4. ff10.1007/978-3-319-91086-4_1ff. ffhal-01887543f.
- [3] Merz, Peter and Freisleben, Bernd. (1997). "A Genetic Local Search Approach to the Quadratic Assignment Problem." Proceedings of the 7th international conference on genetic algorithms. Available at: https://www.researchgate.net/publication/2723845_A_Genetic_Local_Search_Approach_to_the_Quadratic_Assignment_Problem
- [4] Ifeoluwa, "A Genetic Algorithm for Solving Quadratic Assignment Problems (QAP)", Medium (Aug. 23, 2021). Available at: <https://medium.com/codex/a-genetic-algorithm-for-solving-quadratic-assignment-problems-qap-43b8eb82d798>
- [5] Hosein Azarbonyad, Reza Babazadeh, "A Genetic Algorithm for Solving Quadratic Assignment Problem (QAP)." University of Tehran, Iran. Available at: <https://arxiv.org/abs/1405.5050>
- [6] GeeksforGeeks, "Quadratic Assignment Problem (QAP)." (Nov. 17, 2023). Available at: <https://www.geeksforgeeks.org/dsa/quadratic-assignment-problem-qap/>
- [7] QAPLIB, "A Quadratic Assignment Problem Library." Available at: <https://coral.ise.lehigh.edu/data-sets/qaplib/>
- [8] ScienceDirect, "Quadratic Assignment Problem - an Overview". Available at: <https://www.sciencedirect.com/topics/computer-science/quadratic-assignment-problem>
- [9] P. Ji, Yongzhong Wu, Haozhao Liu, "A Solution Method for the Quadratic Assignment Problem (QAP)", The Sixth International Symposium on Operations Research and Its Applications (ISORA'06), Xinjiang, China, August 8–12, 2006. pg. 106–117. Available at: <https://www.aporc.org/LNOR/6/ISORA2006F08.pdf>.
- [10] Zhila Yaseen Taha, Abdulhady Abas Abdullah, Tarik A. Rashid, "Optimizing Feature Selection with Genetic Algorithms: A Review of Methods and Applications", 5 Sep 2024 . Available at: <https://arxiv.org/pdf/2409.14563> [cs.NE] .